

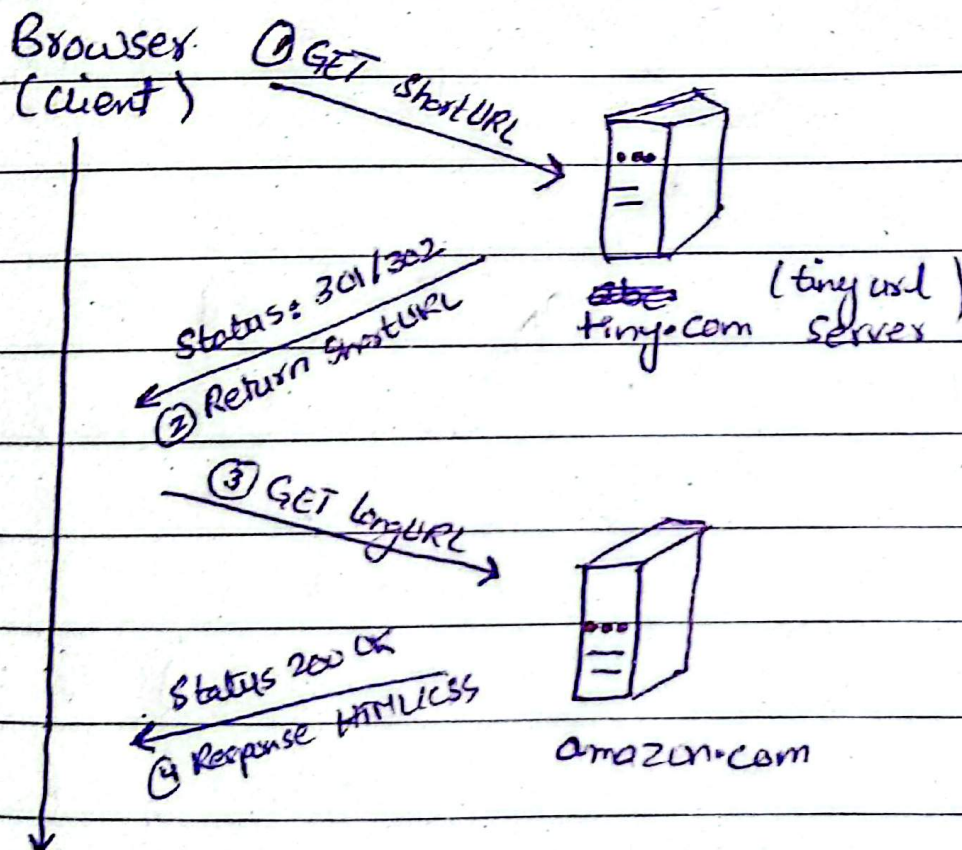
URL Shortener

→ Long Url : `amazon.com/products/1/abc1...1...1?search=...`

→ Short Url : `tiny.com/abc24.com`

↳ No special characters
Easy to remember

→ High Level Design



Date _____

Day _____

→ Decision to Choose 301 & 302

↳ Difference

301 ⇒ Permanent Redirection

↳ Browsers cache the redirect

302 ⇒ Temporary Redirection

↳ Browsers usually don't cache this

↳ More latency because of more hops

↳ Analytics Required

↳ because every request first goes to

bing url server & so we can get analytics from there.

→ Functional Requirements

↳ Long URL → Short URL (memorable)

↳ Unique URL

↳ Length of the URL : eg. (6)

→ why choose 6?

↳ Support for Analytics : YES/NO

↳ if YES then 302

↳ Custom Short URL

↳ YES! ⇒ verify its existence

↳ URL Expiry

Page No. _____

Date _____

Day _____

↳ Custom Branding

↳ Uniqueness in the link itself

↳ /ABC823 (unique)

↳ /BBAC22

↳ repetition

→ Non Function Requirements

↳ Build for 1M customers

↳ High Availability → Fault Tolerance

↳ Scalability

↳ Response : $P(90) = 250ms$

↳ 90% of the customers get 250ms response time

↳ For $<100ms$ ask if it is for all users across globe

YES? would be expensive, multiregion deployment

NO? Only for within region. Deployed in EU and for only EU users $<100ms$

Date _____

Day _____

⇒ Designing the System

① Step #01: Traffic Estimation

↳ 1M Customers

→ Generate links (writers)

→ Consumes links (readers)

↳ This tool will mostly used by social media content creators.

↳ Approx 80% are consumers, 20% creators
80:20

1 Million Users → 80% (R) → 0.8M

↳ 20% (W) → 0.2M

Active Users

(Daily)	(Monthly)	(Yearly)
DAU	MAU	YAU

50% DAU Readers / Writers

↳ So Readers : 80M $\xrightarrow{50\%}$ 40M

Writers : 20M $\xrightarrow{50\%}$ 10M

Helps to understand Sys Architecture

Date _____

Day _____

QPS : Query / Request Per Second

⇒ Writers (DAU) → 0.1 M

→ On Avg a single user: 1 link/day → 0.1 M links/day

↳ For a 5 to 10 year estimate

10 yrs : $0.1 \text{ M} \times 30 \text{ days} \times 12 \text{ months} \times 10 \text{ yrs}$
: 360 M links / 10 yrs

Over 10 yrs period
360 M links would
be stored in our Db

→ QPS : $0.1 \text{ M links/day} \rightarrow 0.1 \text{ M} \times \frac{1}{24 \text{ hrs}} \times \frac{1}{60 \text{ min}} \times \frac{1}{60 \text{ sec}}$

→ $0.1 \text{ M} \div 86400 = 1.157 \text{ QPS}$

≈ 2 QPS Write

⇒ Readers (DAU) → 0.4 M

→ On Avg a single user: 1 link/day → 0.4 M link/day

$\text{QPS} = \frac{0.4 \text{ M}}{86400} = 4.7 \text{ QPS}$

≈ 5 QPS

Page No. _____

Date _____

Day _____

↳ Peak QPS :

↳ It can be 10x, 20x, 2x

↳ Here we assume 2x

↳ Read QPS : $5 \text{ QPS} \times 2 \rightarrow 10 \text{ QPS}$

↳ Write QPS : $2 \text{ QPS} \times 2 \rightarrow 4 \text{ QPS}$

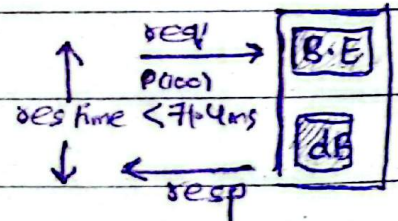
take this as approx. value $\leftarrow \underline{\underline{14 \text{ QPS}}}$

↳ If we have a Single Thread / Single Core system & we want to Achieve 14 QPS.

↳ So it means ek thread ko 14 queries ko process krna hain in 1 sec.

↳ So to process 1 Query it can max take time of 1 = 71.4 ms

14 reqs



q_1

q_2

q_3

q_{14}

| 71.4ms | 71.4ms | 71.4ms | ... | 71.4ms |

← 1000ms →

↳ Agr kisi query par 200ms lag gaye to matter hojagga!

↳ No APIs should do heavy compute

↳ like video processing
JSON parsing

Page No. _____

Date _____

Day _____

(2) Step #02 : Size of ShortURL

memorable $\rightarrow$ Short
 $\rightarrow$ Easy characters

[A-Z][a-z][0-9] No special characters

26 26 10

= 62

1/62 for $n=1$

1/62 x 62 for $n=2$

1/62ⁿ for $n \rightarrow$ total 62ⁿ links

62ⁿ = 300 M $\rightarrow$ estimated from traffic per year

$$\log(62^n) = \log(300M)$$

$$n \log(62) = \log(300M)$$

$$n = \frac{\log(300M)}{\log(62)}$$

$$n \approx 4.73$$

$$\boxed{n \leq 5}$$

62⁵ $\rightarrow$ 916 Million OK

62⁶ $\rightarrow$ 56 Billion future proof (our assumption may be wrong)

62⁷ $\rightarrow$ 3.52 Trillion overkill (so we can choose upper cap)

Page No. _____

Date _____

Day _____

③ Step #03 : Designing Function To Generate URL

↳ Hashing

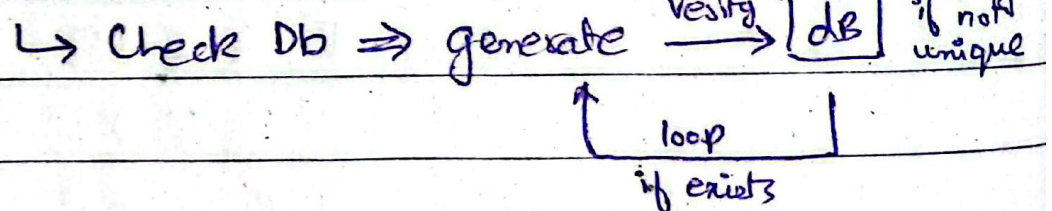
- ↳ SHA256 → 60 + char
 - ↳ CRC32 → 7 + char
 - ↳ MD5 → 32 + char
- } but we want exactly 6 unique ones

↳ Hashing doesnot guarantee uniqueness
↳ We can have collisions

↳ Custom Soln :

↳ Random

↳ Collisions



↳ Incremental

↳ Make a counter
and change values
based on that

Counter

0 — AAAAAA
1 — AAAAAB
2 — AAAAAC
3 —
4 — ZZZZZZ

Helps Cost Analysis of System

Date _____

Day _____

④ Step #04: Storage Calculation (SQL)

↳ Schema Design

Pk: Auto Increment	Short Url	Long URL
—	—	—
10 B	40 B	255 B

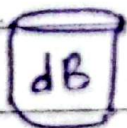
← 300 Byte / Row →

$$300 \text{ M rows} \Rightarrow 300 \text{ M} \times 300 \text{ B}$$

$$300 \times 10^6 \times 300$$

$$90000 \times 10^6$$

$$90 \times 10^9 = 90 \text{ GB}$$



→ # dB case

→ RAM

→ Storage

} AWS / Azure calculators & then
cost nikal sakte per month

Assumption: \$1500 / month

↳ Replica (1500 USD / month)

↳ Multi Region (Let say 2 → 2 × 1500 USD / month)

↳ Total [6000 USD / month]

→ Region → Region Data transfer cost (seperate)

Page No. _____

Date _____

Day _____

• Primary Key Selection

↳ PK must be selected based on data's access pattern

↳ In this case all the search will be based on the Short URL.

↳ Very few usecases where search would be done thru ID - (all short URLs of a user).

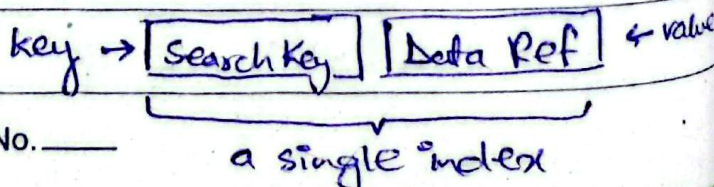
↳ If we assign ID as the PK then the DB will have to scan whole table in each query which will take $O(n)$.

↳ We use Indexes for this Purpose

• Indexing

↳ It is a structure in a database that is used to speed up data retrieval operations by minimizing no. of disk accesses req. to locate record

- Stores copies of selected column (search key) values
- Maintains pointer to the actual data rows in main table
- Allow faster searching



Page No. _____

Date _____

Day _____

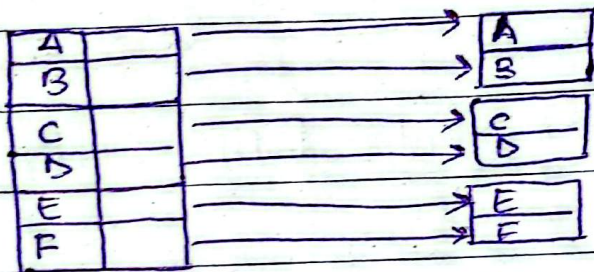
↳ File Organization in Indexes

↳ How indexes are stored Physically

① Sequential (Ordered)

i. Dense Index

↳ One index entry for every search key value



Data File

Index Record

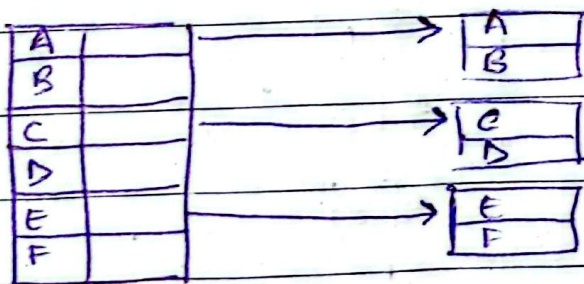
No. of Index Entry =
No. of Db records

→ Greater Performance
→ More Storage

ii. Sparse Index

↳ Each item points to a block

→ less performance
→ less storage



No. of Index Entry =
No. of blocks

→ Here we can only store the block address of 1st element of each block. So we have to only search in a specific block

② Hash File Organization

↳ Uses hash func to map keys to buckets

Date _____

Day _____

• Types Of Indexes

① Clustered Index

can be sparse or dense

efficient range queries

↳ It defines the physical order of rows in a table

↳ Conditions:

↳ The data can be stored in a sequential or sorted manner

↳ The col used as the key must contain unique values

• Single lookup directly in record

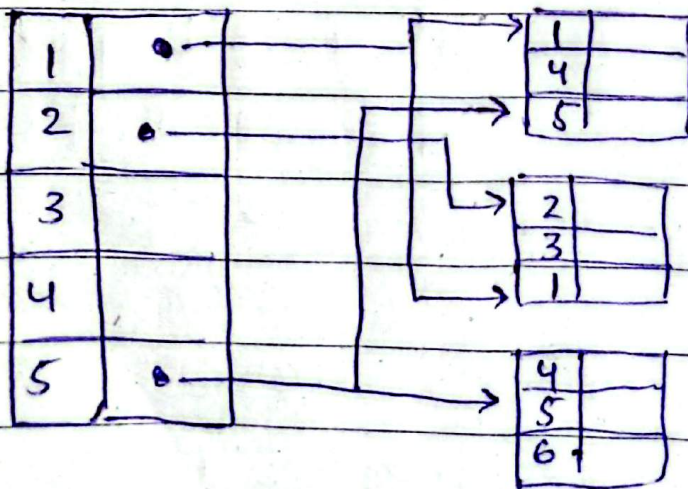
↳ A table can have only ONE clustered index

② Non Clustered Index

→ can only be dense

↳ Physical Order change nhi hota rows ka

↳ Instead we store the ^{record} blocks address where the record is stored



it takes 2 lookup

↳ (1) index file

↳ (2) data record

→ Also called Secondary Index
→ Uses Binary Tree

↳ Multiple non-clustered index can be exist

Page No. _____

Date _____ Clustered Index _____ Non clustered index _____ Day _____

Roll No	Name	Gender
4	Samroze	Male
3	Nasir	Male

NOTE:

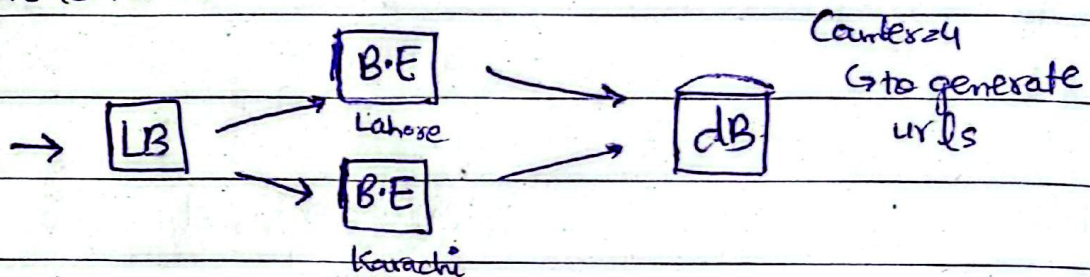
- More indexing slow WRITE operations (insert, update)
- Also increases storage
- High Latency
- PostgreSQL is by default not optimized for write intensive systems

Date _____

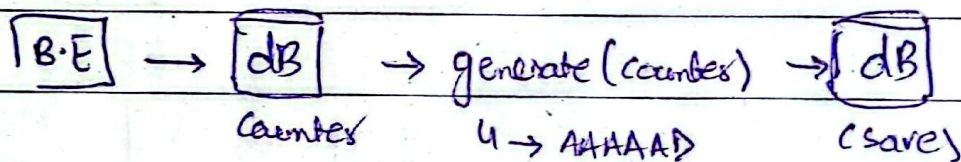
Day _____

(5) Step # 05 : System Design

(i) Approach # 01



• Flow:



• Problems

↳ dB → Single Point Of Failure (SPOF)

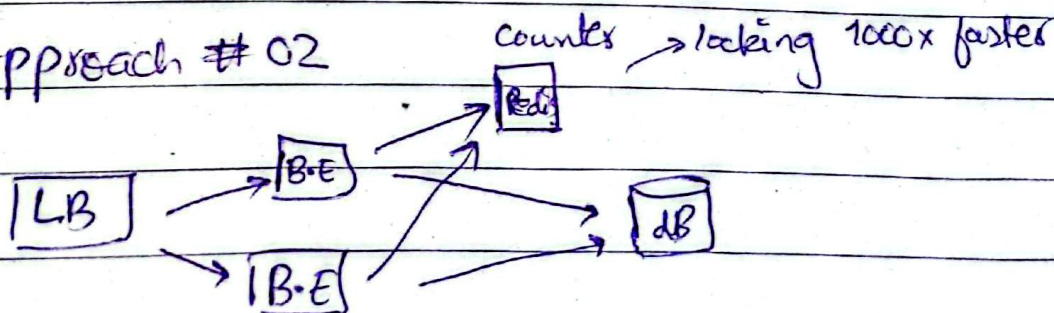
↳ 2 times dB fetch → increase latency → ↑ resp time

↳ RACE CONDITION on counter value

↳ if applied locking then all request have to wait for that lock to open.

↳ ↑ resp time

(ii) Approach # 02



Date _____

Day _____

• Trade Offs

↳ Redis Expensive → tradeoff for performance

↳ dB → SPOF

↳ Redis down → Counter lost

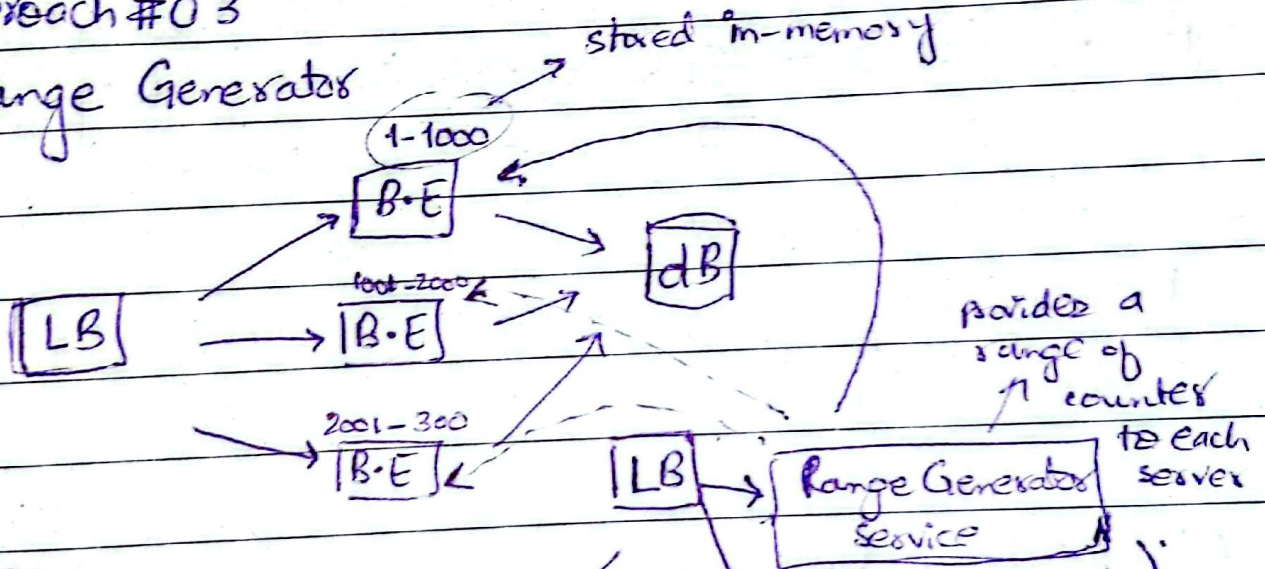
↳ Snapshot Backup

↳ 30 min — 150 new entries

60 min — 300 + +

iii) Approach #03

↳ Range Generator



• Trade Off

↳ if server crashes

it lost his prev

ranges & then gets

new one from

RG service

Added multiple range generator service to ensure high availability

This will give range

b/w 1-150M

150M → 300M

Loss of data tolerable since

we had to make cuts of about

300M & we choose n=6 to about 56 Billion

Page No. _____

Date _____

Day _____

(iv) Approach #04: Cache (Redis) → Frequent URLs

↳ If permanent redirection ON so we can't figure out the popular URLs.

↳ So we would need analytics, so that we only put popular URLs in cache.

→ Rate Limiting

can be any number
usually 1.5

$$\begin{aligned}\# \text{ of Cores} &= \text{QPS} \times \text{resp time} \times \text{safety factor} \\ &= 7 \times 250 \text{ms} \times 1.5 \\ &= 2.625 \approx 3 \text{ core}\end{aligned}$$

↳ means a single server with 4 cores is sufficient!

★ TYPES OF DATABASES

① Relational — organizes data into tables with rows & col
Ex: MySQL, PostgreSQL

② Key Value — Stores data as key value pair for faster lookup
Ex: Redis, Dynamo DB

- (13) Hierarchical — Organizes data in a tree like structure with parent child relationship. Ex: IBM IMS, Windows registry
- (14) Vector — Stores vector data. Ex: ChromaDB
- (15) Embedded — Integrated into application for local storage. Ex: SQLite
- Date _____ Day _____

(3) Document — Manages data in document formats like JSON, suitable for semi-structured data
Ex: MongoDB, CouchDB

(4) Graph — Stores data in nodes and edges, excellent for relationship heavy datasets.
Ex: Neo4j, ArangoDB

(5) Wide-Column — Uses column families to store data, optimized for read/write performance
Ex: Cassandra, HBase

(6) In-memory — Keeps data in RAM for ultra fast access
Ex: Redis, Memcached

(7) Time series — Optimized for sequential, timestamped data entries
Ex: InfluxDB, TimescaleDB

(8) Object Oriented — Stores data as obj. Ex: db4o, ObjectDB

(9) Text Search — Designed for full text search capabilities & indexing
Ex: Elasticsearch, Solr

(10) Spatial — Specialized for geographical data
Ex: PostGIS, Spatialite

(11) Blob — Manages large binary objects (images, videos, doc)
Ex: Amazon S3

(12) Ledger — Immutable data storage. Ex: Hyperledger Fabric